

DOCKER

Hands-On Labs

One-Day Technical Training • 9:00 am – 5:00 pm

Participant Workbook

9 hands-on exercises • Linux (Ubuntu) primary, Windows notes inline

Docker must be installed beforehand — see the Pre-Training Installation Guide

Your name: _____

Contents

How These Labs Work

This is a hands-on Docker course delivered as 9 lab exercises across one day, 9:00 am to 5:00 pm — from your first containers to orchestrating with Docker Swarm. You learn by typing and running commands, not by reading slides. Every exercise has the same shape: an Objective (what you'll achieve), Steps (commands and files to create), and an Expected result (how to confirm it worked). Type the commands yourself — the muscle memory is the point.

WATCH OUT Installing Docker is a PREREQUISITE done before the day, using the separate Pre-Training Installation Guide. We start at 9:00 am with a quick check that everyone's Docker is working, then go straight into the labs. If your setup isn't ready, tell the facilitator at the start.

Self-contained. Every lab uses public Docker Hub images (nginx, ubuntu, php, mysql, wordpress, phpmyadmin, mailhog). Nothing depends on an internal server, so the labs work on any machine with Docker and internet access.

Reading the boxes. Grey monospaced blocks are commands or file contents to type. Green TIP boxes are shortcuts and good habits. Orange WATCH OUT boxes flag common mistakes. Blue WHY IT MATTERS boxes connect a step to the bigger picture.

Platforms. Commands target Linux (Ubuntu). On Windows, run the same commands inside a WSL Ubuntu terminal (Docker Desktop is already installed from the prerequisite) — the Docker CLI is identical. Windows notes are called out inline.

Before the Day — Quick Checklist

- Docker installed and working (you completed the Pre-Training Installation Guide).
- `docker run hello-world` prints "Hello from Docker!" on your machine.
- A free Docker Hub account (hub.docker.com) and a code editor (VS Code recommended).

Schedule — 9:00 am to 5:00 pm

Time	Session
09:00 – 09:15	Welcome & verify your setup
09:15 – 09:45	Docker concepts (image vs container, architecture)
09:45 – 10:25	Lab 1 — Basic Docker commands
10:25 – 11:05	Lab 2 — Docker run & port mapping
11:05 – 11:20	Break
11:20 – 12:00	Lab 3 — Build your first image
12:00 – 12:45	Lab 4 — Build a web app image
12:45 – 13:00	Q&A / buffer
13:00 – 14:00	Lunch
14:00 – 14:40	Lab 5 — Build an image with dependencies
14:40 – 15:20	Lab 6 — Data persistence with volumes
15:15 – 15:30	Break
15:30 – 16:05	Lab 7 — Docker Compose
16:05 – 16:30	Lab 8 — Full WordPress stack
16:30 – 16:55	Lab 9 — Docker Swarm
16:55 – 17:00	Wrap-up & closing

Course Map

Lab	Title	You'll be able to...
1	Basic Docker commands	Run, list, inspect, and manage containers
2	Docker run & port mapping	Serve an app and reach it from the browser
3	Build your first image	Write a Dockerfile and build an image
4	Build a web app image	Use env vars and a custom port in an image
5	Build an image with dependencies	Install packages/extensions in a build
6	Data persistence with volumes	Keep data across container removal
7	Docker Compose basics	Run a multi-service app from one file
8	Compose: full WordPress stack	Wire app + DB + admin with volumes
9	Docker Swarm	Deploy, scale, self-heal, rolling-update

Session 0 — Welcome & Core Concepts

Duration: 30 min (09:00 – 09:45)

First — confirm your setup works

Docker was installed before today. Let's make sure it's ready. Run:

```
docker --version    # prints the installed version
docker run hello-world # prints: Hello from Docker!
```

WATCH OUT If hello-world doesn't run, flag it now. On Linux a 'permission denied' on the Docker socket means you need to log out/in after the group step (or run: newgrp docker). On Windows, make sure Docker Desktop is running (whale icon).

What is Docker?

The problem it solves. Software depends on a specific environment — language version, libraries, OS packages, config. Move it to another machine and those assumptions break. Containers package the app together with everything it needs, so the same artifact behaves identically on a laptop, a server, or the cloud.

Image vs container. An image is a read-only template — a snapshot of a filesystem plus instructions for starting the app. A container is a running (or stopped) instance of an image. Think recipe (image) versus the cooked dish (container): one image can start many containers.

Containers vs virtual machines

Aspect	Virtual Machine	Container
Isolation unit	Full guest OS	Process + filesystem
Startup time	Seconds to minutes	Milliseconds to seconds
Size	Gigabytes	Megabytes
Density	Few per host	Many per host

How Docker is wired together

Three pieces. The Docker client (the docker command) sends your requests to the Docker daemon (dockerd), which builds images and runs containers. Images live in a registry — Docker Hub is the default. When you run docker run, the client asks the daemon, the daemon pulls the image if needed, then starts a container.

```
docker CLI  —> Docker daemon (dockerd) —> Registry (Docker Hub)
(client)      builds / runs / manages      stores images
```

WHY IT MATTERS The image-vs-container distinction underpins everything today — especially Swarm "services," which are just many containers created from one image. If one idea sticks from this session, make it this one.

Lab 1 — Basic Docker Commands

Objective

Get fluent with the everyday commands: running containers, listing them, looking inside, and managing their lifecycle.

Steps

```
# run an interactive Ubuntu container, look around, then leave
docker run -it ubuntu /bin/bash
  cat /etc/os-release
  ls /
  exit

docker ps          # running containers
docker ps -a       # all containers (incl. stopped)
docker images      # local images
docker pull nginx  # download an image without running it
```

Now practise the lifecycle on a named container:

```
docker run -d --name web nginx
docker stop web
docker start web
docker restart web
docker exec -it web bash    # open a shell inside it; exit
docker rm -f web           # stop and remove
```

Command reference

Command	What it does
<code>docker run -it bash</code>	Start a container interactively
<code>docker ps</code> / <code>docker ps -a</code>	List running / all containers
<code>docker images</code>	List local images
<code>docker pull </code>	Download an image
<code>docker stop</code> / <code>start</code> / <code>restart</code>	Control a container's run state
<code>docker exec -it <name> bash</code>	Run a command inside a running container
<code>docker rm <name></code> (<code>rm -f</code>)	Remove a stopped (or running) container

WHY IT MATTERS A container that has exited still exists (`docker ps -a` shows it) and still holds its writable layer until you `docker rm` it. This becomes important when we reach the volumes lab.

Check yourself

1. What's the difference between `docker ps` and `docker ps -a`?
2. How do you open a shell inside a container that's already running?

Lab 2 — Docker Run & Port Mapping

Objective

Run a web server in the background and reach it from your browser by mapping a host port to the container's port.

Steps

```
docker run -d -p 8080:80 --name web nginx
docker ps                # STATUS shows it running
# open http://localhost:8080 in your browser
docker logs web          # see the access log
```

Flags: `-d` runs detached (background); `--name web` gives a friendly name; `-p 8080:80` maps host port 8080 to container port 80. The order is host:container.

Run a second copy on a different host port, then clean up:

```
docker run -d -p 8081:80 --name web2 nginx
# both http://localhost:8080 and http://localhost:8081 work
docker rm -f web web2
```

WATCH OUT Two containers cannot share the same host port. 'port is already allocated' means 8080 is taken — pick another host port (e.g. 8081:80). Remember: host port first, container port second.

Check yourself

3. In `-p 8080:80`, which number is the host and which is the container?
4. Why can't two containers both publish host port 8080?

Lab 3 — Build Your First Image (Dockerfile)

Objective

Package a shell script into your own image using a Dockerfile, then run it.

Steps

Step 1 — In a new folder, create a file named script.sh:

```
#!/bin/bash
echo "Enter something:"
read userInput
echo "You entered: $userInput"
```

Step 2 — In the same folder, create a file named Dockerfile (capital D, no extension):

```
FROM ubuntu
WORKDIR /app
COPY script.sh /app/script.sh
RUN chmod +x /app/script.sh
CMD ["/app/script.sh"]
```

Step 3 — Build and run it:

```
docker build -t myscript:1.0 .
docker run -it myscript:1.0
```

Expected result

The container prints "Enter something:", waits for you to type, and echoes it back. You built and ran your own image.

Concepts: Each Dockerfile instruction (FROM, WORKDIR, COPY, RUN, CMD) adds a layer. The trailing dot in docker build is the build context — the folder sent to the daemon. CMD is the default command run when the container starts.

WATCH OUT Run this one with -it: the script reads from STDIN, so without an interactive terminal it would exit immediately. The Dockerfile must be named exactly 'Dockerfile' unless you pass -f.

Check yourself

5. What does each instruction in a Dockerfile create?
6. What does the . at the end of docker build mean?

Lab 4 — Build a Web App Image (env vars + custom port)

Objective

Containerize a small PHP web page whose background colour is set by an environment variable, served by Apache on a custom port.

Steps

Step 1 — Create index.php:

```
<?php
$name = strtolower(getenv('BACKGROUND_COLOR') ?: 'white');
$map = ['red'=> '#f00', 'green'=> '#0f0', 'blue'=> '#00f', 'yellow'=> '#ff0', 'white'=> '#fff'];
$bg = $map[$name] ?? '#fff';
?>
<!doctype html>
<html><head><style>body{background:<?php echo $bg; ?>;}</style></head>
<body><h1>Hello, Docker!</h1><p>Background from an env var.</p></body></html>
```

Step 2 — Create the Dockerfile (serves on port 5000 instead of 80):

```
FROM php:apache
COPY index.php /var/www/html/
# switch Apache from port 80 to 5000
RUN sed -i 's/80/5000/g' \
    /etc/apache2/sites-available/000-default.conf /etc/apache2/ports.conf
EXPOSE 5000
```

Step 3 — Build and run, mapping a host port to the container's 5000 and passing the colour:

```
docker build -t colorapp:1.0 .
docker run -d -p 8080:5000 -e BACKGROUND_COLOR=blue --name colorapp colorapp:1.0
# open http://localhost:8080 -> blue background

# change the colour by re-running with a different env value:
docker rm -f colorapp
docker run -d -p 8080:5000 -e BACKGROUND_COLOR=green --name colorapp colorapp:1.0
```

WHY IT MATTERS EXPOSE documents the port the app listens on inside the container; -p actually publishes it to the host. -e passes an environment variable the app reads at runtime — the same image produces different output without rebuilding.

Check yourself

7. What's the difference between EXPOSE in the Dockerfile and -p at run time?
8. How did you change the page colour without rebuilding the image?

Lab 5 — Build an Image with Dependencies

Objective

Build a more realistic image that installs system packages and PHP extensions during the build — the kind of Dockerfile a real app needs.

Steps

Reuse the index.php from the previous lab. Create this Dockerfile:

```
FROM php:apache

# system packages needed to build the extensions
RUN apt-get update && apt-get install -y zip libzip-dev

# install PHP extensions into the image
RUN docker-php-ext-install pdo_mysql zip

COPY index.php /var/www/html/

ENTRYPOINT ["apache2-foreground"]
```

Build and run:

```
docker build -t phpapp:1.0 .
docker run -d -p 8080:80 --name phpapp phpapp:1.0
# open http://localhost:8080
```

WATCH OUT Use STRAIGHT double quotes in ENTRYPOINT: ["apache2-foreground"]. Curly/smart quotes (from copying out of a word processor) are invalid JSON and the build or start will fail — a real gotcha.

WHY IT MATTERS Put slow, rarely-changing steps (apt-get install, ext-install) BEFORE COPYing your code. Docker caches each layer, so when you change only index.php, the dependency layers are reused and the rebuild is fast.

Check yourself

9. Why install dependencies before copying application code?
10. What kind of quotes must ENTRYPOINT use, and why?

Lab 6 — Data Persistence with Volumes

Objective

See that container data is disposable by default, then make it survive with a named volume, and share host files with a bind mount.

Steps — prove data is lost without a volume

```
docker run -d --name db1 -e MYSQL_ROOT_PASSWORD=secret mysql:8
# ... MySQL writes its data inside the container ...
docker rm -f db1      # data is gone with the container
```

Steps — keep data with a named volume

```
docker volume create mysqldata
docker run -d --name db2 -e MYSQL_ROOT_PASSWORD=secret \
  -v mysqldata:/var/lib/mysql mysql:8

# remove the container, recreate with the SAME volume:
docker rm -f db2
docker run -d --name db2 -e MYSQL_ROOT_PASSWORD=secret \
  -v mysqldata:/var/lib/mysql mysql:8
# the database is still there – the volume outlived the container
```

Steps — bind-mount a host folder (live files)

```
mkdir -p site && echo '<h1>Hello from a bind mount</h1>' > site/index.html
docker run -d -p 8080:80 -v $(pwd)/site:/usr/share/nginx/html:ro nginx
# open http://localhost:8080 – edit site/index.html and refresh
```

Type	Stored where	Best for
Named volume	Managed by Docker on the host	Databases, persistent app data
Bind mount	A folder you choose on the host	Local dev — live file editing

WATCH OUT

Windows: from WSL use Linux-style paths like \$(pwd)/site and keep files inside the WSL filesystem; from PowerShell use \${PWD}/site. Bind-mounting Windows C:\ paths into WSL is slow.

Check yourself

- Where does data go when you remove a container that has no volume?
- When would you use a bind mount instead of a named volume?

Lab 7 — Docker Compose Basics

Objective

Describe a multi-service application in a single file and bring it up with one command.

Steps

In a new folder, create `compose.yaml` (all public images):

```
services:
  db:
    image: mysql:8
    environment:
      MYSQL_ROOT_PASSWORD: genpass
      MYSQL_DATABASE: appdb
  phpmyadmin:
    image: phpmyadmin
    ports:
      - "8080:80"
    environment:
      PMA_HOST: db
    depends_on:
      - db
  mailhog:
    image: mailhog/mailhog
    ports:
      - "8025:8025"
```

Bring the stack up and explore it:

```
docker compose up -d
docker compose ps
docker compose logs -f db          # Ctrl+C to stop following
# phpMyAdmin: http://localhost:8080 (server: db, user: root, pass: genpass)
# MailHog UI: http://localhost:8025
docker compose down                # stop & remove (add -v to drop volumes)
```

WHY IT MATTERS Compose created one network for the project, so phpmyadmin reaches the database simply as the hostname 'db' — Docker's built-in DNS uses the service name. That's why PMA_HOST is just 'db', no IP address.

WATCH OUT YAML is indentation-sensitive: spaces only, never tabs, and keep it consistent. Modern Docker uses 'docker compose' (a space); the old 'docker-compose' (hyphen) you'll see online is the legacy tool. The 'version:' top line of old files is no longer needed.

Check yourself

13. How does phpMyAdmin find the database without an IP address?
14. Which command starts the whole stack, and which tears it down?

Lab 8 — Compose: A Full WordPress Stack

Objective

Run a complete, real application — WordPress with a MySQL database and phpMyAdmin — using volumes for persistence and a dedicated network.

Steps

Create compose.yaml in a fresh folder:

```
services:
  db:
    image: mysql:8
    environment:
      MYSQL_DATABASE: wpdb
      MYSQL_USER: admin
      MYSQL_PASSWORD: genpass
      MYSQL_ROOT_PASSWORD: genpass
    volumes:
      - db_data:/var/lib/mysql
    networks: [wp]

  wordpress:
    image: wordpress:latest
    depends_on: [db]
    ports:
      - "8888:80"
    environment:
      WORDPRESS_DB_HOST: db
      WORDPRESS_DB_USER: admin
      WORDPRESS_DB_PASSWORD: genpass
      WORDPRESS_DB_NAME: wpdb
    volumes:
      - wp_data:/var/www/html
    networks: [wp]

  phpmyadmin:
    image: phpmyadmin
    ports:
      - "8881:80"
    environment:
      PMA_HOST: db
    networks: [wp]

volumes:
  db_data:
  wp_data:

networks:
  wp:
```

Run it, complete the WordPress installer, then prove persistence:

```
docker compose up -d
# WordPress: http://localhost:8888    (finish the 5-minute install)
# phpMyAdmin: http://localhost:8881

docker compose down          # stops containers but KEEPS the named volumes
```

```
docker compose up -d          # your WordPress site and data are still there
```

WHY IT MATTERS Two volumes do the work: db_data keeps the database, wp_data keeps WordPress files/uploads. Because they're named volumes, docker compose down removes the containers but not the data. (docker compose down -v WOULD delete the volumes.)

WATCH OUT First boot takes a minute while MySQL initialises; WordPress may show a database error until the DB is ready. Wait and refresh. depends_on controls start order, not readiness.

Check yourself

15. Which volumes persist data, and what do they each hold?
16. Why does your site survive a docker compose down (without -v)?

Lab 9 — Docker Swarm

Objective

Turn Docker into an orchestrator: deploy a service with several replicas, scale it, watch it self-heal, and roll out an update with no downtime.

Steps — initialise the swarm & deploy a service

```
docker swarm init          # this machine becomes a manager
docker node ls

docker service create --name web -p 8080:80 --replicas 3 nginx
docker service ls          # desired vs running replicas
docker service ps web      # the 3 individual tasks
# open http://localhost:8080 - Swarm load-balances across the replicas
```

Multi-node (reference): On a real cluster the manager runs `docker swarm init --advertise-addr <MANAGER-IP>` and prints a join command; each worker runs `docker swarm join --token <TOKEN> <MANAGER-IP>:2377`. A single-node swarm is enough to learn every concept here.

Steps — scale & self-heal

```
docker service scale web=5
docker service ps web      # now 5 tasks

docker ps                  # copy ONE web container's ID
docker rm -f <container_id> # kill a replica
docker service ps web      # Swarm has already started a replacement
```

WHY IT MATTERS You declared a desired state — 5 replicas. When you killed one, Swarm noticed reality no longer matched and started a new task to get back to 5, with no action from you. That self-healing is the whole point of an orchestrator.

Steps — rolling update & rollback

```
docker service update --image nginx:1.27 \
  --update-parallelism 1 --update-delay 10s web
docker service ps web      # tasks update a batch at a time
docker service rollback web # undo to the previous image
```

Cleanup

```
docker service rm web
docker swarm leave --force # leave the single-node swarm
```

WATCH OUT `docker run` does NOT use the swarm — it always starts a plain local container. Orchestrated work uses `docker service` (and `docker stack` for Compose-style files).

Check yourself

17. What's the difference between a service and a task?
18. You set 4 replicas and kill one container — what does Swarm do, and why?
19. Which command performs a rolling update, and how do you undo it?

Wrap-Up, Cleanup & Next Steps

What you built today

In one day you ran and managed containers, mapped ports, and built three of your own images — from a simple script to a PHP web app with dependencies. You then made data survive with volumes, orchestrated multi-container apps with Compose (including a full WordPress stack), and finished with Docker Swarm — deploying, scaling, self-healing, and rolling out updates across a cluster.

Cleanup commands

```
docker swarm leave --force      # if a swarm is still active
docker compose down -v          # in any compose folder you used
docker rm -f $(docker ps -aq)    # remove all containers
docker system prune -a --volumes # reclaim disk (images, networks, volumes)
```

WATCH OUT `docker system prune -a --volumes` deletes ALL unused images and volumes — perfect for a clean teardown, but never run it casually on a shared or work machine.

Where to go next

- Multi-stage builds to shrink production images.
- Image security: scanning, minimal/official base images, running as non-root.
- `docker stack deploy` to run Compose files on a Swarm.
- Kubernetes as a dedicated, more advanced course once Swarm feels comfortable.
- Official docs: docs.docker.com • Free practice: labs.play-with-docker.com.

— End of Participant Workbook —